

Module 9: ORDER BY Clause

The **ORDER BY** clause is used to sort query results in ascending or descending order. It is one of the most commonly used SQL clauses in reporting, dashboards, analytics, and business intelligence.



Introduction

When data is retrieved from a database, SQL does **not guarantee any order** unless an **ORDER BY** clause is explicitly used.

Example:

Without sorting:

EmployeeID	EmployeeName	Salary
103	Aman	60000
101	Kunal	45000
102	Rahul	55000

This order may change every time the query runs.

To control the order of results, we use:

ORDER BY



Learning Objectives

After completing this module, you will be able to:

- ✓ Sort records in ascending order
- ✓ Sort records in descending order
- ✓ Sort using multiple columns
- ✓ Sort text, numbers, and dates
- ✓ Use aliases in sorting
- ✓ Sort calculated columns
- ✓ Understand NULL sorting behavior

✓ Apply sorting in real-world reports



Table of Contents

1. What is ORDER BY?
2. ORDER BY Syntax
3. Ascending Order (ASC)
4. Descending Order (DESC)
5. Sorting Text Data
6. Sorting Date Data
7. Sorting Multiple Columns
8. ORDER BY Using Column Position
9. ORDER BY Using Aliases
10. ORDER BY with Calculated Columns
11. Sorting NULL Values
12. Query Execution Order
13. Common Mistakes
14. Best Practices
15. Summary
16. Practice Questions

1 What is ORDER BY?

The **ORDER BY** clause sorts the result set returned by a query.

Example

Table:

EmployeeID	EmployeeName	Salary
103	Aman	60000
101	Kunal	45000
102	Rahul	55000

Query:

```
SELECT *  
FROM Employees  
ORDER BY Salary;
```

Output:

EmployeeName	Salary
Kunal	45000
Rahul	55000
Aman	60000

Why ORDER BY is Important

Used in:

- Dashboards
- Reports
- Ranking systems
- Analytics
- Data exports

2 ORDER BY Syntax

Basic syntax:

```
SELECT ColumnName
FROM TableName
ORDER BY ColumnName;
```

Example

```
SELECT *
FROM Employees
ORDER BY EmployeeName;
```

General Structure

```
SELECT
FROM
WHERE
ORDER BY
```

3 Ascending Order (ASC)

Ascending means:

```
Smallest → Largest  
A → Z  
Oldest → Newest
```

Syntax

```
ORDER BY ColumnName ASC
```

Example

```
SELECT *  
FROM Employees  
ORDER BY Salary ASC;
```

Output:

EmployeeName	Salary
Kunal	45000
Rahul	55000
Aman	60000

ASC is Default

These queries are identical:

```
SELECT *  
FROM Employees  
ORDER BY Salary;
```

```
SELECT *  
FROM Employees  
ORDER BY Salary ASC;
```



Descending Order (DESC)

Descending means:

```
Largest → Smallest  
Z → A  
Newest → Oldest
```

Syntax

```
ORDER BY ColumnName DESC
```

Example

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```

Output:

EmployeeName	Salary
Aman	60000
Rahul	55000
Kunal	45000

Real-World Examples

Highest salaries:

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```

Latest orders:

```
SELECT *  
FROM Orders  
ORDER BY OrderDate DESC;
```

5 Sorting Text Data

Text columns can also be sorted.

Example

Table:

EmployeeName
Rahul
Aman
Kunal

Query:

```
SELECT *  
FROM Employees  
ORDER BY EmployeeName ASC;
```

Output:

```
Aman  
Kunal  
Rahul
```

Descending

```
SELECT *  
FROM Employees  
ORDER BY EmployeeName DESC;
```

Output:

```
Rahul  
Kunal  
Aman
```

Alphabetical Sorting

ASC **DESC**

A → Z Z → A

Sorting Date Data

Dates can be sorted chronologically.

Example

EmployeeName	JoiningDate
Kunal	2024-05-01
Rahul	2022-03-10
Aman	2025-01-15

Oldest First

```
SELECT *  
FROM Employees  
ORDER BY JoiningDate ASC;
```

Newest First

```
SELECT *  
FROM Employees  
ORDER BY JoiningDate DESC;
```

Common Business Uses

- Recent Orders
- New Customers
- Latest Transactions
- Employee Joining History

7 Sorting Multiple Columns

You can sort by more than one column.

Syntax

```
ORDER BY Column1,  
        Column2;
```

Example

Table:

EmployeeName	Department	Salary
Kunal	IT	50000
Rahul	IT	45000
Aman	HR	60000

Query:

```
SELECT *  
FROM Employees
```

```
ORDER BY Department,  
Salary DESC;
```

Execution:

Step 1

Sort by Department

```
HR  
IT
```

Step 2

Within each department:

Sort Salary DESC

Output:

EmployeeName	Department	Salary
Aman	HR	60000
Kunal	IT	50000
Rahul	IT	45000

Mixed Sorting

```
ORDER BY Department ASC,  
Salary DESC;
```

Very common in reporting.



ORDER BY Using Column Position

SQL allows sorting using column numbers.

Example:

```
SELECT EmployeeID,  
       EmployeeName,  
       Salary  
FROM Employees  
ORDER BY 3;
```

Meaning:

```
ORDER BY Salary;
```

Why Avoid This?

If query changes:

```
SELECT EmployeeName,  
       Salary
```

Then:

```
ORDER BY 3
```

causes errors.

Best Practice

Use column names instead.



ORDER BY Using Aliases

Example

```
SELECT EmployeeName,  
       Salary * 12 AS AnnualSalary
```

```
FROM Employees
ORDER BY AnnualSalary DESC;
```

Output:

Highest annual salary first.

Benefits

- ✓ Cleaner queries
 - ✓ Easier maintenance
-

10 ORDER BY with Calculated Columns

You can sort using calculations.

Example

```
SELECT ProductName,
       Price,
       Price * 1.18 AS FinalPrice
FROM Products
ORDER BY FinalPrice DESC;
```

Output:

Products ordered by final selling price.

Example

```
SELECT EmployeeName,
       Salary,
       Salary * 12 AS AnnualSalary
FROM Employees
ORDER BY Salary * 12 DESC;
```

1 1 Sorting NULL Values

Consider:

EmployeeName	Salary
Kunal	45000
Rahul	NULL
Aman	60000

Query:

```
SELECT *  
FROM Employees  
ORDER BY Salary;
```

SQL Server generally places:

```
NULL values first
```

when sorting ascending.

Result:

```
NULL  
45000  
60000
```

Handling NULLs

Example:

```
SELECT *  
FROM Employees  
WHERE Salary IS NOT NULL  
ORDER BY Salary;
```

1 2 Query Execution Order

Query:

```
SELECT EmployeeName,  
        Salary  
FROM Employees  
WHERE Salary > 40000  
ORDER BY Salary DESC;
```

Actual Execution:

1. FROM
2. WHERE
3. SELECT
4. ORDER BY

Understanding execution order helps:

- Debug queries
- Optimize performance
- Learn advanced SQL

1 3 Common Mistakes

Misspelled Column Names

Bad:

```
ORDER BY Salery
```

Error:

```
Invalid column name
```

Sorting by Non-Selected Alias

Bad:

```
SELECT EmployeeName
FROM Employees
ORDER BY AnnualSalary;
```

Error.

Using Column Positions

Bad:

```
ORDER BY 3
```

Better:

```
ORDER BY Salary
```

Assuming Data is Sorted

Bad:

```
SELECT *
FROM Employees;
```

Never assume order.

Always use:

```
ORDER BY
```

1

4

Best Practices

Always Use ORDER BY When Order Matters

Good:

```
ORDER BY Salary DESC
```

Use Column Names

Avoid:

```
ORDER BY 2
```

Sort Only When Needed

Sorting large datasets consumes resources.

Use Meaningful Aliases

Example:

```
AnnualSalary
```

instead of:

```
Col1
```

Combine with TOP

Common pattern:

```
SELECT TOP 10 *  
FROM Employees  
ORDER BY Salary DESC;
```

Real-World Example

Top 5 Highest Paid Employees

```
SELECT TOP 5
    EmployeeID,
    EmployeeName,
    Salary
FROM Employees
ORDER BY Salary DESC;
```

Employee Salary Report

```
SELECT EmployeeName,
    Salary,
    Salary * 12 AS AnnualSalary
FROM Employees
ORDER BY AnnualSalary DESC;
```



Summary

In this module, you learned:

- ✓ ORDER BY Clause
 - ✓ ASC Sorting
 - ✓ DESC Sorting
 - ✓ Text Sorting
 - ✓ Date Sorting
 - ✓ Multi-Column Sorting
 - ✓ Aliases in ORDER BY
 - ✓ Calculated Column Sorting
 - ✓ NULL Sorting
 - ✓ Query Execution Order
-



Practice Questions

Theory

1. What is ORDER BY?
 2. What is ASC?
 3. What is DESC?
 4. Why is ORDER BY important?
 5. Can ORDER BY sort text data?
 6. Can ORDER BY sort dates?
 7. How does multi-column sorting work?
 8. Why avoid column positions?
 9. How are NULL values sorted?
 10. What is the execution order involving ORDER BY?
-

Practical Exercises

Task 1

Display all employees sorted by:

Salary **ASC**

Task 2

Display all employees sorted by:

Salary **DESC**

Task 3

Sort employees alphabetically by name.

Task 4

Sort employees by:

Department **ASC**
Salary **DESC**

Task 5

Create:

AnnualSalary

and sort by it.

Task 6

Display:

TOP 5

highest-paid employees.

Challenge Project

Using an Employees table:

Generate reports for:

- Highest-paid employees
- Newest employees
- Department-wise salary rankings
- Employees sorted by annual salary

Use:

- ORDER BY
 - ASC
 - DESC
 - Multiple columns
 - Calculated columns
-



Next Module

→ Module 10: Aggregate Functions

Topics Covered:

- COUNT()
- SUM()
- AVG()

- MIN()
- MAX()
- Aggregate Function Rules
- NULL Handling in Aggregates
- Business Analytics Examples